Pytracer 2: Honest Tiered Instrumentation for Numerical Variability Profiling in Python

Yohan Chatelain
yohan.chatelain@gmail.com

Draft of July 16, 2026

Abstract

Floating-point instability in Python scientific code is easy to create and hard to see: a numerically fragile formulation returns a plausible number, and the NumPy/SciPy/scikit-learn stack executes most of its arithmetic behind C, Cython, and BLAS boundaries that Python-level tools cannot observe. We present Pytracer 2, a numerical variability profiler that runs a program repeatedly under a perturbed environment, records the inputs and outputs of numerical calls, aligns the runs call-by-call, and attributes the loss of significant bits to specific functions and callsites. Pytracer 2 is a clean-slate rewrite of PyTracer built around two forms of honesty. *Coverage honesty*: instrumentation is organized as five composable tiers with declared contracts (boundary patching, ufunc proxies, tainted tracer arrays, a PEP 669 runtime monitor, and an `LD_PRELOAD` BLAS census), and every report states which numerical work was observed and which was provably missed. *Metric honesty*: element-wise significant digits are never conflated with the optimistic sig(mean) summary proxy; every metric row carries its basis. We evaluate Pytracer 2 with an eight-part experiment campaign. It detects and correctly ranks 8/8 classical numerical pathologies; worst-case microbenchmark overhead ranges from 7.2× (monitor tier) to 320.7× (taint tier) while the original PyTracer no longer traces at all on any current Python; on an adversarial permutation workload the summary proxy reports 52.75 stable bits where the element-wise measurement finds 1.89; and an end-to-end scikit-learn case study localizes a PCA instability to `numpy.linalg.eigh`, with the `check` and `diff` commands turning the finding into a failing CI gate and a detected float32 regression. Pytracer 2 is open source and all measurements in this paper are generated by re-runnable scripts.

1 Introduction

Python is the dominant orchestration language of computational science, but the arithmetic it orchestrates is opaque. A call to `numpy.linalg.solve` descends through C dispatch into LAPACK; a scikit-learn `fit` runs Cython loops that call BLAS kernels no Python-level tool ever sees. When results carry fewer correct digits than their fifteen printed decimals suggest — because of catastrophic cancellation, ill-conditioning, or the accumulation of rounding error — nothing in the standard workflow reveals it. Studies of production scientific pipelines have shown that such silent variability is large enough to change scientific conclusions, for example in neuroimaging, where perturbations on the order of machine error propagate to qualitatively different brain-network statistics [14].

Stochastic arithmetic gives an empirical handle on the problem: run the program several times while randomizing the rounding of floating-point operations (Monte Carlo Arithmetic [16],

implemented by tools such as Verificarlo [6] and Verrou [8]), and the spread of results across runs estimates the number of significant digits [21]. But the perturbation engine answers only *whether* the final output is stable. The question users actually have is *where* instability originates: which function destroys precision, which merely propagates noisy inputs, and which changed the control flow of the program.

PyTracer [4] pioneered answering that question for Python: it instrumented modules at import time, recorded the inputs and outputs of targeted functions across perturbed runs, and visualized per-call significant digits. Its architecture — trace, parse, visualize — was sound, but a detailed technical review of the implementation identified structural problems: a global monkey-patch of `builtins.type` and `isinstance` that changed the semantics of every library in the traced process; silent trace truncation when runs diverged; trace files read back through `dill` pickles and `eval`; error handling in which recoverable conditions terminated the traced application while data-integrity failures were silently swallowed; and a packaging setup pinned to a 2021 stack with no declared dependencies. In Section 5.2 we show experimentally that the original PyTracer can no longer trace even a three-line NumPy program on any currently supported Python interpreter.

This paper presents PYTRACER 2, a clean-slate rewrite, and an experimental evaluation of its claims. Two design principles distinguish it from both its predecessor and other dynamic analyses:

Coverage honesty. No single interception mechanism can observe all numerical work in a Python process (Section 3.1), so PYTRACER 2 composes five instrumentation tiers, each with an explicit contract of what it sees, what it is blind to, and what it costs. The uninstrumented remainder is not ignored: a runtime monitor censuses the C callables that executed without a wrapper, and an `LD_PRELOAD` shim censuses the BLAS kernels invoked from compiled code. Every report states what fraction of the numerical work was observed. Absence of an instability signal is never presented as evidence of stability.

Metric honesty. The cheap cross-run statistic — the stability of an argument's mean, sig(mean) — is an optimistic proxy: element permutations across runs are nearly invisible to it (Section 5.7 measures a 50.86-bit gap on an adversarial workload). PYTRACER 2 computes true element-wise significant digits from stored array payloads whenever possible and labels every metric row with its basis, so the proxy can never masquerade as the measurement.

This paper makes the following contributions:

- a tiered instrumentation architecture for Python numerical tracing — boundary patching, ufunc proxies, tainted tracer arrays, a PEP 669 runtime monitor, and a native BLAS census — with declared, measured contracts (Section 3);
- digit-loss *attribution*: per aligned call, the amplification metric amp = $\text{sig}_{\min}(\text{inputs}) - \text{sig}_{\min}(\text{outputs})$ separates functions that destroy precision from functions that merely receive noisy inputs (Section 3.5);
- a numerical-CI workflow: `pytracer check` gates on significance thresholds with a nonzero exit code, and `pytracer diff` compares two experiments to catch numerical regressions across library or precision changes (Section 5.8);

- an eight-part experiment campaign, fully scripted and regenerated from raw measurements: detection efficacy on a gallery of 8 classical pathologies (8/8 detected and correctly ranked), per-tier overhead, coverage accounting, alignment robustness under control-flow divergence, the summary-proxy gap, an end-to-end scikit-learn case study, storage and crash-safety, and a head-to-head installability comparison with PyTracer 1 (Section 5).

PYTRACER 2 deliberately does *not* perturb arithmetic. It is the observability layer above perturbation engines such as Verificarlo, Verrou, or a fuzzy libmath environment: they create the variability; PYTRACER 2 localizes and attributes it. The same separation makes PYTRACER 2 noise-model agnostic — the experiments in this paper use random input perturbation, and the identical workflow under a Monte Carlo Arithmetic backend measures true floating-point instability (Section 5.1).

2 Background

Stochastic arithmetic and significant digits. IEEE-754 arithmetic commits to one rounding per operation, so a single execution reveals nothing about the sensitivity of its result to rounding. Monte Carlo Arithmetic (MCA) [16] replaces each operation's result with a randomly rounded value at a chosen virtual precision; across repeated executions, the empirical distribution of any intermediate or final value reflects its numerical quality. The number of significant bits of a value x observed across n runs can be estimated as $\mathrm{sig}(x) = -\log_2(\sigma/|\mu|)$ for sample mean μ and standard deviation σ, with rigorous confidence intervals given by Sohier et al. [21]; PYTRACER 2 delegates this estimation to the `significantdigits` package [2], which implements both the centered-normality-hypothesis and general variants. Practical MCA implementations include Verificarlo [6], an LLVM compiler pass with interchangeable backends, and Verrou [8], a Valgrind-based tool requiring no recompilation; CADNA [12] implements the related CESTAC method with three concurrent executions. Randomly-rounded system math libraries ("fuzzy" environments) extend the perturbation to `libm` calls and have been used to quantify uncertainty in large neuroimaging pipelines [14].

Profiling versus perturbing. A perturbation engine varies the arithmetic; a profiler observes the consequences. Keeping the two separate is a deliberate design position. First, perturbation is language- and ABI-specific (an LLVM pass, a Valgrind tool, a preloaded libm), whereas observation of a Python program is a Python-level concern. Second, agnosticism to the noise model lets the same profiler serve MCA, random rounding, input perturbation, or domain-specific structured noise [4]. PYTRACER 2 orchestrates the repeated runs (rotating backend seeds through per-run environment variables), records what happened at every observed call, and computes cross-run metrics; whether the runs differ because of MCA or because of jittered inputs is a property of the experiment, recorded in its metadata, not of the tool.

Why observing Python numerics is hard. Python's flexibility makes interception deceptively easy in the common case and structurally incomplete in general. An ordinary function such as `numpy.linalg.solve` can be wrapped by attribute patching. But NumPy *ufuncs* (`numpy.add`, `numpy.exp`, ...) are C objects carrying semantically distinct methods (`reduce`, `accumulate`, `outer`, `at`); user code checks `isinstance(f, np.ufunc)`; and — critically — operator dispatch (`a + b`) never reads the `numpy.add` module attribute at all, going instead through the ndarray C

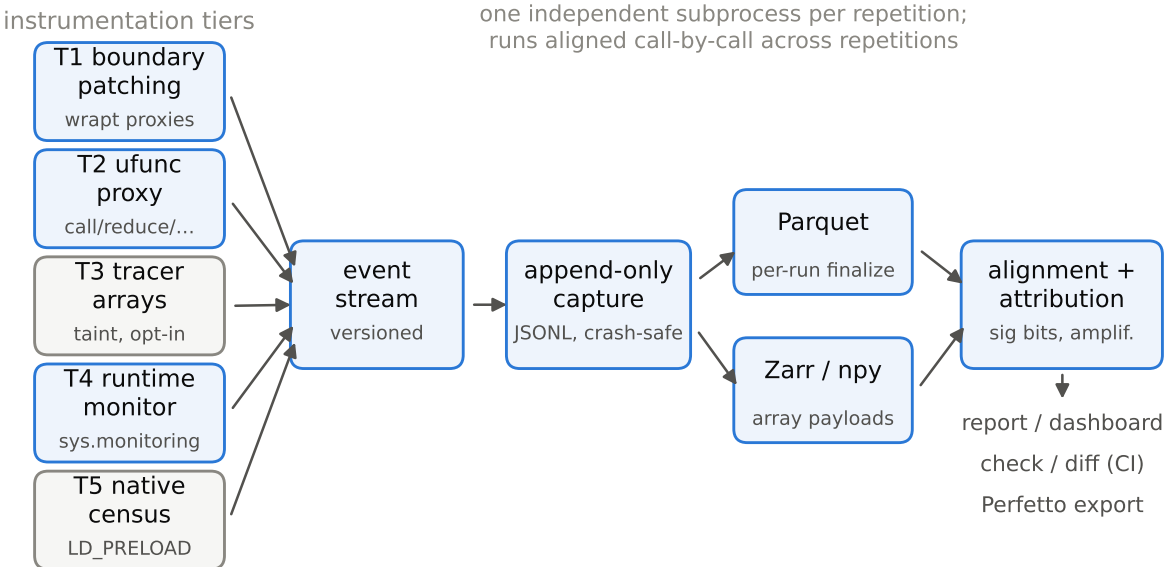

Figure 1: PYTRACER 2 architecture. Five instrumentation tiers (left) feed a schema-versioned event stream captured append-only (crash-safe JSONL), finalized per run to Parquet with array payloads in Zarr. Runs are aligned call-by-call and aggregated into digit-loss attribution, consumed by reports, the dashboard, CI gates, and a Perfetto export.

type slots. Aliases captured before patching (`from numpy import sum`) keep the raw function forever. Deeper still, scikit-learn's Cython code calls BLAS through `scipy.linalg.cython_blas`: those `dgemm` invocations never touch a Python object. Any honest Python tracer must therefore either declare these blind spots or eliminate them with progressively more invasive mechanisms — PYTRACER 2 does both, tier by tier, and measures what remains (Section 3).

3 Design

Figure 1 shows the pipeline: instrumentation tiers emit a schema-versioned event stream; events are captured append-only and finalized to columnar storage; runs are aligned call-by-call; aggregation attributes digit loss to functions; and the results feed reports, an interactive dashboard, and CI gates.

3.1 The instrumentation problem, stated honestly

The central tension of dynamic numerical tracing is *transparency* (observe every numerical operation without the user changing code) versus *specificity* (each interception mechanism works only for particular kinds of callables, and the aggressive mechanisms alter the semantics of the traced program). PyTracer 1 resolved the tension in favor of transparency by patching `builtins.type` and `isinstance` process-wide so that wrapped objects would masquerade as their originals — the cautionary tale that motivates this design: a tracer must not change the meaning of the program it observes.

PYTRACER 2 instead composes five tiers, each a separate module with a declared contract

Table 1: The tier contract. "Sees" and "blind to" are declared in documentation and enforced by the conformance test suite; overheads are measured in Section 5.4.

Tier	Sees	Blind to	Overhead	Risk	Default
T1 patch	targeted Python functions/methods	aliases, operators, C-internal	low	low	on
T2 ufunc	targeted ufuncs incl. `reduce`, `accumulate`, `out=`	operator dispatch, C-internal	low–med	low	on
T3 tracer array	all NumPy ops on tainted data, alias-proof	taint stripped at `asarray`/C entry	high	med	off
T4 monitor	call graph, C-call occurrence, line coverage	operand values	med	low	on
T5 native census	BLAS/LAPACK kernels + shapes, incl. from Cython	operand values, non-BLAS C	low	low–med	off

(Table 1). The default configuration (`hybrid` = T1+T2+T4) is safe: it never proxies a returned value, never alters a type observable by user code, and its blind spots are enumerated by the deeper tiers and the coverage report.

T1 — boundary patching wraps targeted Python-level functions and methods (plugin lists plus user `-target` globs) using `wrapt` proxies [7], which transparently forward `__name__`, signatures, and `isinstance` via `__class__`. The wrapped call emits input events, calls the original, emits output events, and returns the result object *unchanged* — never copied, never proxied.

T2 — ufunc interception answers the ufunc specificity problem with a dedicated `UfuncProxy`: it intercepts `__call__`, `reduce`, `reduceat`, `accumulate`, `outer`, and `at` — each tagged with the method name, since a `reduce` is numerically a different operation than an elementwise call — handles `out=` buffers (summarized after the call, flagged in-place), and keeps `isinstance(numpy.add, np.ufunc)` true. Its declared blind spot is operator dispatch, covered by T3.

T3 — tracer arrays (opt-in, `-instrument taint`) close the operator gap for selected data: values tagged with `pytracer.taint(x)`, plus outputs of traced calls, become `TracedArray` instances whose `__array_ufunc__` and `__array_function__` protocols observe *every* NumPy operation on them — including `a + b` and `a @ b` and calls made from third-party Python code — regardless of aliasing, because protocol dispatch keys on the operand type, not on how the function was looked up. Taint is stripped by `np.asarray` at many C entry points, so T1 wrappers re-taint their outputs to re-seed propagation. T3 alters `type(x)` checks and is therefore never a default: it is the drill-down instrument for a region T1/T2 already localized.

T4 — runtime monitor uses `sys.monitoring` (PEP 669 [19]; hence the Python $\geq$ 3.12 floor) to record structure only: the call graph that gives every event its parent call, per-line execution counts whose cross-run differences localize control-flow divergence, and — centrally for coverage honesty — a census of calls to C callables that had *no* wrapper installed. The monitor cannot see operand values, but it can prove that `numpy.mean` ran 2,000 times unobserved, which is exactly what the coverage report and the `suggest-targets` workflow need.

T5 — native kernel census (opt-in, `-native`, Linux) is a small `LD_PRELOAD` shim interposing BLAS/LAPACK dynamic symbols (`dgemm_`, `dgesv_`, ..., including the prefixed ILP64

symbols shipped in NumPy/SciPy wheels). It records which kernels ran and their dimensions — including calls from Cython that never touch a Python object — but not operand values: it is a coverage and attribution instrument, not a value tracer. Value-level perturbation of these kernels is the perturbation engine's job.

3.2 Execution model

Each repetition is an independent subprocess: in-process repeats would share module caches, RNG state, and perturbation-backend seeds, and one crash would kill the experiment. The child installs instrumentation *before* importing the target script, executes it as `__main__`, and finalizes its trace in a `finally` block. Per-run environment overrides from the `[perturb.env]` config table (with `{run_index}` substitution) rotate MCA seeds for engines like Verificarlo. Run metadata captures an *allowlisted* subset of the environment only (`PATH`, `OMP_*`, `VFC_*`, ...) — never the raw environment, so secrets cannot leak into shareable trace directories.

3.3 Events, storage, and crash-safety

Every event is a schema-versioned `msgspec` struct: run, call and parent-call identifiers, tier, callsite, per-callsite occurrence counter, phase (input/output/exception), and a numeric summary (dtype, shape, moments, norms, NaN/Inf/subnormal counts, and a fingerprint). During capture, events are appended as JSON lines — a crash loses at most the final partial line. At run finalization the capture is converted to `events.parquet` for analysis and interoperability (pandas/duckdb); array payloads go to Zarr with zstd compression under the `store_arrays` policy. Parquet is deliberately *not* written during capture: row-group buffering plus a crash yields truncated files, whereas the JSONL capture of a SIGKILLed run remains readable (measured in Section 5.9). No pickle format appears anywhere in the system.

3.4 Alignment

Cross-run metrics require knowing which call in run 3 corresponds to which call in run 7. The unit of alignment is the *call* (aligning input and output events independently mismatches them when counts diverge), keyed by module, qualified name, callsite, ufunc method, and a deterministic per-callsite occurrence counter. Three modes: `strict` requires identical call sequences and fails loudly at the first divergence (doubling as the determinism test); `callsite` (default) matches by callsite key and occurrence index; `fuzzy` performs per-callsite longest-common-subsequence matching using input shapes and dtypes as hints, so unmatched calls become missing/extra records instead of cascading misalignment. The alignment report also carries the T4 line-count divergence table, which localizes control-flow divergence even inside untraced code.

3.5 Metrics: digit-loss attribution

For every aligned call group, PYTRACER 2 computes significant bits of each input and output across runs, on one of two bases, never conflated: *element* — true element-wise significant digits computed from stored array payloads (the real measurement); or *summary* — sig(mean), the cross-run stability of the argument's mean, used when payloads were not stored and labeled as the optimistic proxy it is. Every derived metric row carries its `sig_basis`.

The headline metric is per-call *amplification*:

$$\text{amp} \;=\; \text{sig}_{\min}(\text{inputs}) \;-\; \text{sig}_{\min}(\text{outputs}), \tag{1}$$

the bits of precision destroyed by the call itself. Ranking functions by amplification first, absolute output significance second, and control-flow divergence third separates the *sources* of instability from its *victims* — the question users actually have. NaN/Inf instability (an output that is NaN in some runs but not others) is flagged independently.

3.6 Coverage accounting and the discovery loop

Coverage analysis merges (a) events observed per tier, (b) the T4 census of C callables invoked with no wrapper — which doubles as the `suggest-targets` list — and (c) T5 native kernels with no correlated traced call. The report renders this as an observed/unobserved section with two structural notes that are always present: operator dispatch on ndarrays is not observable at T1/T2, and calls inside compiled extensions are invisible to every Python tier. The discovery loop is then: run cheaply with the monitor, let `suggest-targets` propose wrappers for what it saw, re-run with those targets, and drill into localized regions with taint mode.

3.7 Numerical CI

`pytracer check` evaluates an experiment against thresholds (minimum significant bits, maximum divergence, NaN/Inf policies; also read from `pytracer-thresholds.toml`) and exits nonzero on violation, making stability a regression-testable property. `pytracer diff` compares two experiments function-by-function and, with `-fail-on-regression`, gates on significance drops beyond a tolerance — the A/B instrument for library upgrades, precision changes, and flag flips (exercised in Section 5.8).

4 Implementation

PYTRACER 2 is roughly 6,000 lines of Python (plus a 190-line C shim for the native census) organized as `instrumentation`, `trace`, `storage`, `analysis`, `report`, `dashboard`, and `cli` packages. Core dependencies are deliberately few: NumPy, `wrapt`, `msgspec`, PyArrow, and Jinja2; SciPy, scikit-learn, Zarr, `significantdigits`, and Dash are optional — a tracer must not constrain the versions of the libraries it studies. Python $\geq$ 3.12 is required by the PEP 669 monitor.

Several implementation rules are enforced by CI rather than convention. `import pytracer` in an empty directory with no configuration succeeds silently and creates nothing (PyTracer 1 opened cache directories, log files, and reconfigured the root logger at import time). No `sys.exit` exists outside the `cli` package; the library raises `PytracerError` subclasses, and a tracing failure degrades to "this call not recorded" rather than terminating the traced application. Every artifact carries a schema version. The `@pytracer.trace_function` decorator lets users instrument their own functions explicitly; it is a no-op outside `pytracer run`, so decorated code runs unmodified everywhere.

Transparency of the wrappers is tested, not assumed: the conformance suite asserts that patched functions preserve results bit-for-bit against unpatched runs, that `isinstance(numpy.add, np.ufunc)` holds under the T2 proxy, that unpatching restores the exact original attributes (including inherited methods patched onto subclasses), and that the occurrence counters used for alignment are deterministic under threads. The examples gallery doubles as a detection regression suite: CI asserts that the unstable variant of every classical pathology loses significantly more bits than its stable twin — the tool must keep detecting what it exists to detect.

The T5 shim is compiled on demand with the system C compiler and interposes both the standard BLAS/LAPACK symbols and the prefixed ILP64 symbols of NumPy/SciPy wheels; `pytracer doctor` reports which BLAS is loaded and whether interposition is feasible, alongside checks for optional dependencies, config validity, target resolvability, and the presence of a perturbation backend.

Reports are generated as Markdown, self-contained HTML, and JSON; the optional Dash dashboard adds an overview, a per-call explorer with element-wise drill-down, a call timeline colored by amplification, and the coverage view. `pytracer export` emits a Chrome-tracing/Perfetto timeline with significance annotations.

5 Evaluation

We evaluate PYTRACER 2 through eight experiments, each answering one research question:

E0 Does PyTracer 1 still work on current toolchains (the motivation for the rewrite)? (§5.2)

E1 Does PYTRACER 2 detect and correctly rank classical numerical pathologies? (§5.3)

E2 What does each instrumentation tier cost? (§5.4)

E3 What does coverage accounting reveal that a conventional tracer would hide? (§5.5)

E4 How do the alignment strategies behave under control-flow divergence? (§5.6)

E5 How optimistic is the sig(mean) proxy relative to element-wise significant digits? (§5.7)

E6 Does the end-to-end workflow localize a real instability in a scikit-learn pipeline, and do the CI gates fire? (§5.8)

E7 Is the capture format crash-safe, and what does storage cost? (§5.9)

5.1 Setup and perturbation model

All experiments ran in a single Linux x86-64 container with CPython 3.13.12, NumPy 2.5.1, SciPy 1.18.0, and scikit-learn 1.9.0 (OpenBLAS backend). Every number, table, and figure in this section is produced by a script in the repository's `paper/experiments/` directory, writing raw CSV results from which the tables and figures are generated — the paper's numbers cannot drift from the measurements.

No stochastic-arithmetic backend is installed in this environment, so run-to-run variability is induced by *unseeded random input perturbation*: each repetition draws a small random jitter (or, in the pathology gallery, fresh random inputs) exactly as the project's continuous-integration suite does. We state this plainly, in the spirit of the tool's own metric honesty: the experiments measure the *conditioning* of the traced computations under input perturbation and the correctness of PYTRACER 2's attribution machinery, not the rounding-level stability of a fixed input. The identical workflow run inside a Verificarlo or fuzzy-libmath container — PYTRACER 2 rotates the MCA seed per run through its `[perturb.env]` mechanism — measures true floating-point instability with no change to any command in this section. Detection experiments use 5 repetitions per experiment (20 for alignment, 10 for the case study).

Table 2: E0 — PyTracer 1 (final archived revision) on modern interpreters, with modern versions of its undeclared dependencies installed and its default configuration supplied. "Trace" attempts a three-line NumPy program.

Python	Install	Import	CLI	Trace	First failure
3.10	✓	✓	✓	×	`AttributeError`
3.11	✓	✓	✓	×	`TypeError`
3.12	✓	✓	✓	×	`TypeError`
3.13	✓	✓	✓	×	`TypeError`

5.2 E0: PyTracer 1 on modern toolchains

We installed PyTracer 1 from its final archived revision into fresh virtual environments for CPython 3.10–3.13, together with modern versions of its (undeclared) dependencies, and probed four stages: install, import, CLI startup, and tracing a three-line NumPy program (Table 2). Installation succeeds — the package declares no dependencies at all, so there is nothing to conflict — and a bare import immediately exits because a `PYTRACER_CONFIG` environment variable is demanded at import time. With the shipped default configuration supplied, the CLI starts, but *tracing fails on every interpreter*: on 3.10 the import-hook machinery trips over a removed private NumPy attribute (`numpy.lib.format._MAX_HEADER_SIZE`), and on 3.11–3.13 the global `builtins.type` replacement itself fails (`TypeError: type '_Type' is not subscriptable`) — the highest-risk design element breaking before any tracing begins. The rewrite was not a stylistic preference; the original tool is inoperable on every currently supported Python.

5.3 E1: detection efficacy on the pathology gallery

PYTRACER 2 ships a gallery of eight classical numerical-accuracy pathologies, each pairing an unstable formulation with its stable fix in one script: catastrophic cancellation in the textbook variance formula, the naive quadratic formula for the small root, running summation versus Shewchuk's exact `fsum` [13, 20], an ill-conditioned Hilbert solve (condition number $\sim 10^{13}$), the finite-difference step-size dilemma, naive `expm1`/`log1p`, and expanded polynomial evaluation near a root [9, 11]. We traced each script with 5 repetitions and full array storage, and asked two questions: does the unstable variant measure substantially fewer significant bits than its stable twin, and does the amplification ranking place it above the twin?

Table 3 and Figure 2 show the result: 8/8 pathologies detected, with gaps from 4.7 to 32.9 bits, and the unstable variant ranked above its stable twin in all eight cases. Two readings deserve comment. First, the quadratic-roots and polynomial cases collapse to ≈ 0 significant bits — total loss — while their stable rewrites retain 25–33 bits under the same input noise: the formulation, not the data, destroys the precision, and the amplification column says so directly (32.9 bits destroyed by `naive_small_root` versus -0.0 by its fix). Second, summation-order shows the smallest gap (4.7 bits): the workload sums the *same* 20,000 values in a per-run shuffled order, so the naive running sum varies purely through rounding non-associativity — a small effect honestly measured (48.3 bits retained) against the exactly rounded, order-invariant `fsum` control at the full 53 bits.

Table 3: E1 — detection on the pathology gallery (`-repeat 5`, element-wise basis where arrays were stored). $\text{sig}_{\min}$ is the minimum output significant bits across runs; Gap is stable − unstable; Amp. is the amplification (Eq. 1) of the unstable variant; Rank is unstable / stable position in the per-script instability ranking.

Pathology	Formulation	$\text{sig}_{\min}$ (bits)		Gap	Amp.	Rank
		unstable	stable	(bits)	(bits)	
cancellation	`naive_variance`	0.1	5.88	5.79	25.09	1 / 2
quadratic roots	`naive_small_root`	0.0	32.89	32.89	32.89	1 / 2
summation order	`naive_running_sum`	48.3	52.96	4.66	-48.3	1 / 2
ill conditioned solve	`solve_hilbert`	19.39	38.94	19.55	20.03	1 / 2
finite differences	`forward_diff_tiny_h`	9.96	36.81	26.85	30.97	1 / 2
expm1 log1p	`naive_expm1`	12.46	26.83	14.37	14.37	2 / 4
expm1 log1p	`naive_log1p`	12.01	26.83	14.83	14.83	1 / 3
polynomial eval	`expanded_poly`	0.08	25.46	25.37	41.47	1 / 2

Table 4: E2 — worst-case microbenchmark overhead per tier (median of 5). Events/s is the sustained event rate; B/event the finalized capture size per event.

Workload	Tier	Overhead	Events	Events/s	B/event
20k tiny numpy.sum calls	T1	105.5×	40,000	10,457	483
2M-element numpy.sum calls	T1	46.9×	200	49	510
20k tiny numpy.sum calls	T2	113.0×	100,000	23,870	432
operator loop (x*a+b)	T3	320.7×	30,000	11,284	559
end-to-end simple_numpy.py	T4	7.2×	–	–	–

5.4 E2: per-tier overhead

Table 4 and Figure 3 report worst-case microbenchmarks (median of 5 measurements) in which the traced operation itself costs ∼2 µs, so per-event cost dominates: many tiny `numpy.sum` calls under T1 (105.5×), the same calls through the T2 ufunc proxy (113.0×), large-array calls where summary computation dominates (46.9×), an operator loop under T3 taint (320.7×), and an end-to-end script under the T4 monitor alone (7.2×, subprocess startup included).

Three observations. The per-call cost is the summaries: each traced call computes moments, norms, and counts over its operands twice (inputs and outputs) — the measurements *are* the product, and `mode = "metadata"` exists for users who want call structure without them. Taint mode's 320.7× is by design the drill-down tier, applied to a region a cheaper tier already localized, never to a whole program. And the tier whose job is discovery — the T4 monitor — costs only 7.2× end-to-end, which is what makes the discover-then-target workflow (§3.6) practical. Real workloads with meaningful compute per call sit far below these ratios (the large-array workload already halves the tiny-call ratio while tracing 200× fewer events).

5.5 E3: coverage honesty

We traced three workloads (a NumPy micro-script, an ill-conditioned linear solve, and the scikit-learn pipeline of §5.8) under three configurations: T1+T2 only (`patch`), the default `hybrid` (adds the T4 monitor), and `hybrid` plus the T5 native census. Table 5 reports what each configuration

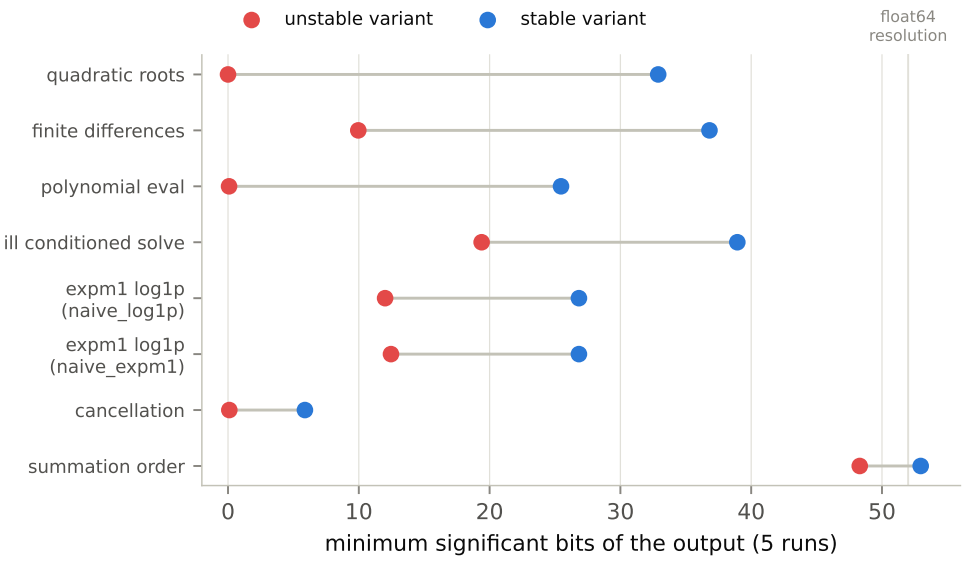

Figure 2: E1 — minimum output significant bits for the unstable and stable formulation of each pathology. The gap is the detection signal; the ranking (Table 3) is the attribution.

traced and — the point of the experiment — what it *admits* it did not trace.

The `patch` rows are the cautionary tale: with no monitor, every workload reports *zero* untraced callables — not because coverage is complete but because the tracer cannot see what it misses. Turning on the monitor reveals 1–4 numerical callables running unobserved (`numpy.mean`, `numpy.hstack`, estimator constructors); adding `-native` shows the BLAS kernels the linear solve and the sklearn pipeline execute below Python. Closing the loop, `suggest-targets` proposed four targets for the sklearn pipeline; re-running with them traced the one genuinely patchable numerical function among them (`numpy.hstack`, $5 \to 6$ traced), while the three remaining suggestions are estimator class constructors that the census correctly reports and the patcher correctly declines to wrap. The report also always carries the two structural blind spots as explicit notes: operator dispatch and compiled-extension internals.

5.6 E4: alignment under control-flow divergence

We traced two adversarial programs with 20 repetitions: one whose branch on an unseeded random draw changes which calls execute, and one whose traced call count itself is a random loop length, then re-aligned the same traces under each strategy (Table 6).

`strict` refuses both traces outright, with an error naming the exact callsite and the runs missing it — the correct behavior for its role as determinism check. `callsite` and `fuzzy` both recover the aligned prefix (3 of 7 call groups on the branch program, 9 of 25 on the random loop) and report the remainder as divergent rather than silently misattributing them — the failure mode that PyTracer 1's `zip`-based merging exhibited, where the shortest run silently truncated all others. Divergent call groups are not discarded: they feed the divergence score that ranks control-flow-unstable functions, and the T4 line-count table localizes *where* the flow diverged even in untraced code.

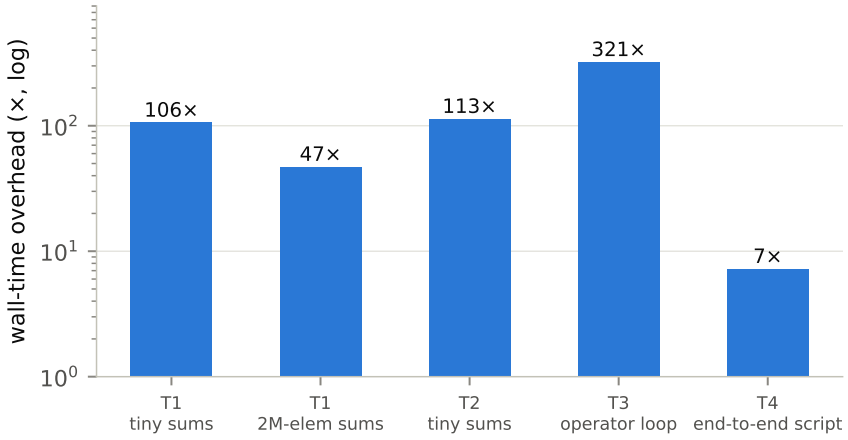

Figure 3: E2 — wall-time overhead by tier (log scale). These are deliberate worst cases: the traced operation costs ∼2 µs, so the ratios bound the per-event cost rather than describe typical workloads.

5.7 E5: the summary-proxy gap

Every workload from a subset of the gallery, plus one adversarial program, was traced twice with identical settings except `-store-arrays`: `never` forces the sig(mean) summary basis, `always` enables the element-wise basis. Table 7 and Figure 5 compare the two per function.

For well-behaved functions the proxy tracks the element-wise measurement within a few bits — and for the mildly ill-conditioned Hilbert solve it overstates stability by 3–4 bits. The adversarial case makes the design point: a function returning the same multiset of values in a different order each run has permutation-invariant summary statistics, so the proxy reports 52.75 bits — numerically indistinguishable from perfect stability — while the element-wise measurement finds 1.89 bits: the value at any given position is essentially random. The gap of 50.86 bits is why PYTRACER 2 stores array payloads by default where feasible, computes element-wise digits as the primary metric, and stamps every row that had to fall back to the proxy with `sig_basis = "summary"`.

5.8 E6: end-to-end case study — a scikit-learn pipeline

The case study is a standard pipeline — `StandardScaler`, 8-component PCA, logistic regression — on a fixed synthetic dataset of 400 samples and 12 features in which one feature nearly duplicates another (separation 10^{-4}), so exactly one pair of covariance eigenvalues is nearly degenerate. Each of 10 repetitions applies a relative input jitter of 3×10^{-7}, chosen to be representable in both float64 and float32. We traced the pipeline with the NumPy and sklearn plugins, then repeated the identical experiment with the data cast to float32 (Table 8).

PYTRACER 2 localizes the instability precisely: `numpy.linalg.eigh` — PCA's eigendecomposition — retains only 4.3 bits with an amplification of 12.2 bits, and the instability propagates to `PCA.transform` (10.0 bits). The physics is textbook: the near-degenerate eigenpair makes the corresponding eigenvectors extremely sensitive to perturbation [11]; PYTRACER 2's contribution is that the ranking finds it automatically in a pipeline the user did not annotate, and distinguishes the source (`eigh`, high amplification) from downstream victims.

Table 5: E3 — coverage accounting across tier configurations. "Untraced seen" counts numerical callables (and their call counts) that executed with no wrapper — observable only when the T4 monitor is active. "Native kernels" are distinct BLAS/LAPACK symbols recorded by the T5 shim.

Workload	Configuration	Traced	Untraced seen		Native
		fns	callables	calls	kernels
numpy micro	patch	2	0	0	0
	hybrid	2	1	3	0
	hybrid+native	2	1	3	0
linear solve	patch	3	0	0	0
	hybrid	3	3	12	0
	hybrid+native	3	3	12	1
sklearn pipeline	patch	5	0	0	0
	hybrid	5	4	12	0
	hybrid+native	5	4	12	1
	hybrid+suggested	6	3	9	0

Table 6: E4 — alignment strategies on control-flow-divergent traces (20 runs, identical trace re-analyzed per strategy). `strict` aborts by design, naming the first divergent callsite.

Workload	Strategy	Call groups	Matched	Divergent	Outcome
random branch	strict	–	–	–	aborts
	callsite	7	3	4	ok
	fuzzy	7	3	4	ok
random-length loop	strict	–	–	–	aborts
	callsite	25	9	16	ok
	fuzzy	25	9	16	ok

The two CI gates then fire as designed. `pytracer check -min-sig-bits 20 -max-divergence 0.01` exits 1 (failure): this pipeline does not meet a 20-bit floor, and a numerical CI would correctly block it. `pytracer diff` between the float64 and float32 experiments with `-fail-on-regression` exits 1, reporting three regressions beyond its 1-bit tolerance — `eigh` drops from 4.3 to 2.1 bits and `PCA.transform` from 10.0 to 8.1 — turning a precision downgrade that changes no test assertion into a caught regression.

5.9 E7: storage footprint and crash-safety

Table 9 reports per-run storage on a call-sparse workload and an event-heavy one (2,000 traced calls). For the heavy trace the append-only JSONL capture costs 6.7 MiB while its finalized Parquet is 158 KiB — 44× smaller — justifying the two-format design: JSONL exists to survive crashes, Parquet to be queried.

Crash-safety was tested destructively: a workload that SIGKILLs its own process — no cleanup, no `atexit` — on repetitions 4–6 of a 6-repetition experiment run with `-continue-on-error`. All 3 killed runs retain readable JSONL captures (each missing at most a trailing partial line), the experiment completes, and analysis recovers events from all 6 runs, killed included. The same scenario under a capture format with buffered row groups would have truncated the killed runs' traces entirely.

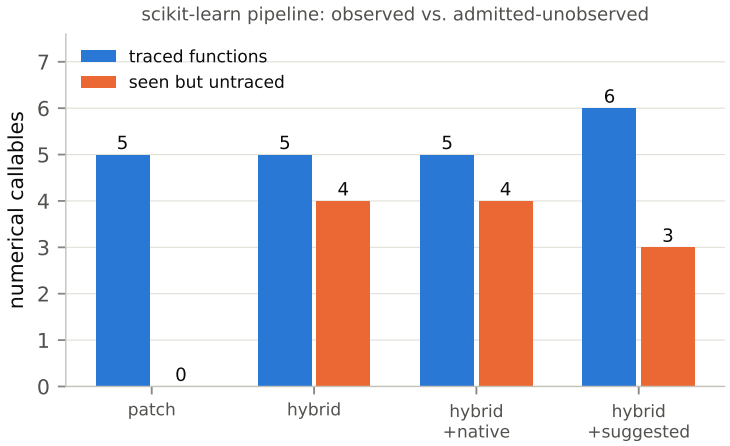

Figure 4: E3 — the scikit-learn pipeline: traced functions vs. admitted-untraced numerical callables per configuration. The `patch` configuration misses the same work — it just cannot report it.

5.10 Threats to validity

The perturbation model is input jitter rather than rounding-level MCA; detection results therefore demonstrate correct *attribution of conditioning*, and transfer to MCA measurements by construction of the workflow rather than by measurement here. Overhead microbenchmarks bound worst-case per-event cost on one container and CPU; absolute ratios will vary with BLAS, CPU, and workload granularity. The E0 matrix supplies modern versions of PyTracer 1's undeclared dependencies; a period-exact 2021 environment would run it, which is precisely the point being made. Case-study numbers depend on the synthetic dataset's conditioning; the scripts expose every constant for independent variation.

6 Related Work

Perturbation engines. Verificarlo [6] instruments floating-point operations at LLVM compile time with interchangeable backends (MCA, cancellation detection); Verrou [8] achieves the same dynamically under Valgrind without recompilation; CADNA [12] implements CESTAC with synchronous triplicated execution. Fuzzy system libraries randomize `libm` itself, enabling container-level perturbation of unmodified pipelines, an approach used to quantify condition-level uncertainty in neuroimaging analyses [14]. PyTracer 2 sits deliberately above all of these: they make runs vary, it explains where the variation came from. File-based localization [17] attributes pipeline variability at the granularity of intermediate files; PyTracer 2 works at the granularity of function calls within one process.

Numerical debugging and repair. FpDebug [1] shadows every operation with high-precision values under Valgrind; Herbgrind [18] tracks error back to root causes in binaries; Herbie [15] rewrites floating-point expressions for accuracy, and FPTaylor [22] bounds round-off error of small expressions rigorously; FPBench [5] standardizes their benchmarks. These tools analyze expressions or compiled binaries; none observes the Python/C boundary where scientific Python

Table 7: E5 — sig(mean) summary proxy vs. element-wise significant bits for the same functions under the same perturbation. Positive proxy optimism means the proxy overstates stability.

Workload	Function	sig(summary) (bits)	sig(element) (bits)	Proxy optimism (bits)
permuted output	`permuted_pipeline`	52.75	1.89	50.86
	`stable_pipeline`	53.0	52.71	0.29
cancellation	`naive_variance`	1.0	0.0	1.0
	`stable_variance`	6.96	6.28	0.68
summation order	`exact_fsum`	52.96	52.96	0.0
	`naive_running_sum`	47.3	48.92	-1.61
	`pairwise_sum`	52.62	51.96	0.66
ill-conditioned solve	`solve_hilbert`	23.05	19.66	3.4
	`solve_well_conditioned`	42.19	39.36	2.83
finite differences	`central_diff_optimal_h`	37.1	36.81	0.29
	`forward_diff_tiny_h`	10.25	10.25	-0.0

Table 8: E6 — scikit-learn case study (10 runs, element-wise basis): minimum and median output significant bits per traced call, float64 vs. float32 pipeline, and float64 amplification. Rows without values are calls whose outputs are non-numeric (fitted-estimator objects).

Traced call	float64 (bits)		float32 (bits)		Amp.
	min	median	min	median	(bits)
`numpy.linalg.eigh`	4.26	4.26	2.14	2.14	12.22
`sklearn.decomposition._pca.PCA.transform`	9.98	9.98	8.13	8.13	3.13
`numpy.add.reduce`	0.0	21.27	0.0	21.26	13.11
`sklearn.decomposition._pca.PCA.fit`	–	–	–	–	–
`sklearn.linear_model._logistic.LogisticRegression.fit`	–	–	–	–	–

programs actually spend their numerical effort, and none addresses the coverage question — what fraction of a NumPy/sklearn workload the analysis saw — that PYTRACER 2 makes a first-class output.

Tracing numerical behavior in context. VeriTracer [3] added temporal, contextualized traces of significance to Verificarlo at the C/Fortran level. PyTracer [4] brought instability profiling to Python via import hooks and is this work's direct predecessor; Section 5.2 quantifies its current state, and Sections 3–4 describe how PYTRACER 2 replaces its global `builtins` patching with tiered, contracted instrumentation, its pickle trace format with schema-versioned columnar storage, and its silent truncation with explicit alignment. The PEP 669 monitoring API [19] that PYTRACER 2 builds on was designed for low-overhead tooling and, to our knowledge, PYTRACER 2 is the first numerical QA tool built on it. NumPy's array protocols (`__array_ufunc__`, `__array_function__`) [10] are the mechanism behind tier T3, used elsewhere for units, autodiff, and lazy evaluation; PYTRACER 2 repurposes them for alias-proof observation.

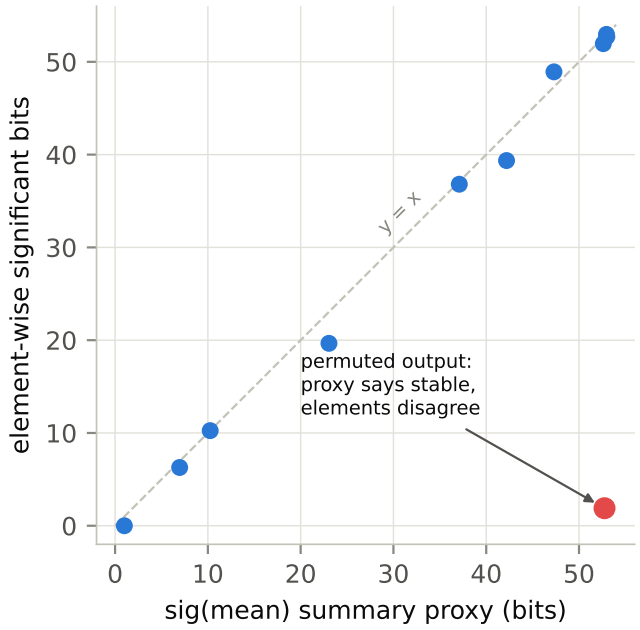

Figure 5: E5 — element-wise significant bits against the sig(mean) proxy. Points on the diagonal are cases where the proxy is honest; the permuted-output workload (red) is the failure mode that motivates the two-basis design.

Table 9: E7 — per-run storage (mean of 5 runs, full array storage). Arrays are Zarr/npy payloads enabling element-wise significance.

Workload	JSONL	Parquet	Arrays	Parquet/JSONL
linear solve (few calls)	8 KiB	10 KiB	7 KiB	1.26×
2k-call loop	6.7 MiB	158 KiB	3.9 MiB	0.02×

7 Conclusion

PyTracer 2 rebuilds Python numerical-variability profiling on two commitments its predecessor could not keep: say what you did not observe, and never let a proxy impersonate a measurement. The tiered instrumentation stack makes the transparency/specificity trade-off explicit and composable — safe boundary patching and ufunc proxies by default, tainted tracer arrays and a native BLAS census when the question warrants them, and a PEP 669 monitor that turns every blind spot into a reported census rather than silence. The experiments show the result is not just safer but more capable: 8/8 classical pathologies detected and correctly attributed, control-flow-divergent runs aligned rather than silently truncated, a 50.86-bit proxy blind spot measured and closed by element-wise significance, a real PCA instability localized to `numpy.linalg.eigh` in an unannotated scikit-learn pipeline, and stability enforced as a CI property through `check` and `diff` — all while the original PyTracer no longer traces a three-line program on any current interpreter.

Three directions follow. Tracing currently observes the parent process only; a child-bootstrap hook would extend coverage to joblib and multiprocessing workers. The tier architecture invites

backends beyond NumPy — PyTorch's `__torch_function__` and JAX's tracing machinery are natural T3 analogues. And the discovery loop would benefit from closing the last gap between census and wrapper: correlating T5 kernel timestamps with the T4 call timeline to attribute native FLOPs to the exact Python call that triggered them. All code, experiment scripts, and the raw results behind every number in this paper are available in the PYTRACER 2 repository.

References

[1] Florian Benz, Andreas Hildebrandt, and Sebastian Hack. A dynamic program analysis to find floating-point accuracy problems. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 453–462, 2012. doi: 10.1145/2254064.2254118.

[2] Yohan Chatelain and Pablo de Oliveira Castro. significantdigits: Compute significant digits of stochastic computations. `https://github.com/verificarlo/significantdigits`, 2024.

[3] Yohan Chatelain, Pablo de Oliveira Castro, Eric Petit, David Defour, Jordan Bieder, and Marc Torrent. VeriTracer: Context-enriched tracer for floating-point arithmetic analysis. In *2018 IEEE 25th Symposium on Computer Arithmetic (ARITH)*, pages 61–68, 2018. doi: 10.1109/ARITH.2018.8464687.

[4] Yohan Chatelain, Nigel Yong, Gregory Kiar, and Tristan Glatard. PyTracer: Automatically profiling numerical instabilities in Python. *IEEE Transactions on Computers*, 72(6):1792–1803, 2023. doi: 10.1109/TC.2022.3225929.

[5] Nasrine Damouche, Matthieu Martel, Pavel Panchekha, Chen Qiu, Alexander Sanchez-Stern, and Zachary Tatlock. Toward a standard benchmark format and suite for floating-point analysis. In *9th International Workshop on Numerical Software Verification (NSV)*, pages 63–77, 2017. doi: 10.1007/978-3-319-54292-8_6.

[6] Christophe Denis, Pablo de Oliveira Castro, and Eric Petit. Verificarlo: Checking floating point accuracy through Monte Carlo arithmetic. In *2016 IEEE 23rd Symposium on Computer Arithmetic (ARITH)*, pages 55–62, 2016. doi: 10.1109/ARITH.2016.31.

[7] Graham Dumpleton. wrapt: A Python module for decorators, wrappers and monkey patching. `https://github.com/GrahamDumpleton/wrapt`, 2024.

[8] François Févotte and Bruno Lathuilière. Studying the numerical quality of an industrial computing code: A case study on code_aster. In *10th International Workshop on Numerical Software Verification (NSV)*, pages 61–80, 2017. doi: 10.1007/978-3-319-63501-9_5.

[9] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991. doi: 10.1145/103162.103163.

[10] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[11] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002. doi: 10.1137/1.9780898718027.

[12] Fabienne Jézéquel and Jean-Marie Chesneaux. CADNA: A library for estimating round-off error propagation. *Computer Physics Communications*, 178(12):933–955, 2008. doi: 10.1016/j.cpc.2008.02.003.

[13] William Kahan. Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1):40, 1965. doi: 10.1145/363707.363723.

[14] Gregory Kiar, Yohan Chatelain, Pablo de Oliveira Castro, Eric Petit, Ariel Rokem, Gaël Varoquaux, Bratislav Misic, Alan C. Evans, and Tristan Glatard. Numerical uncertainty in analytical pipelines lead to impactful variability in brain networks. *PLOS ONE*, 16(11): e0250755, 2021. doi: 10.1371/journal.pone.0250755.

[15] Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. Automatically improving accuracy for floating point expressions. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1–11, 2015. doi: 10.1145/2737924.2737959.

[16] Douglass Stott Parker. Monte Carlo arithmetic: Exploiting randomness in floating-point arithmetic. Technical Report CSD-970002, University of California, Los Angeles, Computer Science Department, 1997.

[17] Ali Salari, Gregory Kiar, Lindsay Lewis, Alan C. Evans, and Tristan Glatard. File-based localization of numerical perturbations in data analysis pipelines. *GigaScience*, 9(12), 2020. doi: 10.1093/gigascience/giaa106.

[18] Alex Sanchez-Stern, Pavel Panchekha, Sorin Lerner, and Zachary Tatlock. Finding root causes of floating point error. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 256–269, 2018. doi: 10.1145/3192366.3192411.

[19] Mark Shannon. PEP 669 – low impact monitoring for CPython. `https://peps.python.org/pep-0669/`, 2021.

[20] Jonathan Richard Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363, 1997. doi: 10.1007/PL00009321.

[21] Devan Sohier, Pablo de Oliveira Castro, François Févotte, Bruno Lathuilière, Eric Petit, and Olivier Jamond. Confidence intervals for stochastic arithmetic. *ACM Transactions on Mathematical Software*, 47(2):1–33, 2021. doi: 10.1145/3432184.

[22] Alexey Solovyev, Charles Jacobsen, Zvonimir Rakamarić, and Ganesh Gopalakrishnan. Rigorous estimation of floating-point round-off errors with symbolic Taylor expansions. In *International Symposium on Formal Methods (FM)*, pages 532–550, 2015. doi: 10.1007/978-3-319-19249-9_33.